Core Principle

The app model should own the state the UI cares about.

Services may hold internal state, but they should not be the primary source of truth for UI behavior unless they represent a clear, stateful subsystem.

Model vs Service Responsibilities

Model state represents what the user sees and can act on.

Services interact with external systems and may track operational details like retries, queues, or inflight work.

Only the service's public, user-relevant state should be exposed upward.

Example: Stateful Service with Observable State

This service owns a clear state machine representing sync progress.

Internal mechanics are hidden; consumers only see high-level state.

```
actor SyncService {
    enum State: Sendable {
        case idle
        case syncing(Double?)        // optional progress
        case failed(String)
        case upToDate(Date)
    }

    private var state: State = .idle
    private var continuations: [AsyncStream<State>.Continuation] = []

    func states() -> AsyncStream<State> {
        AsyncStream { continuation in
            continuations.append(continuation)
            continuation.yield(state)
        }
    }

    private func publish(_ newState: State) {
        state = newState
        continuations.forEach { $0.yield(newState) }
    }

    func runSync() async {
        publish(.syncing(nil))
```

```
        // perform sync work
        publish(.upToDate(Date()))
    }
}
```

Example: App Model Composing the Service

The app model observes the service and exposes state the UI can bind to.

The model does not duplicate service internals—only its public contract.

```
@Observable
final class AppModel {
    var syncState: SyncService.State = .idle
    private let sync: SyncService

    init(sync: SyncService) {
        self.sync = sync
        Task { await observeSync() }
    }

    private func observeSync() async {
        for await state in sync.states() {
            self.syncState = state
        }
    }

    func userTappedSync() {
        Task { await sync.runSync() }
    }
}
```

Why This Works

There is exactly one source of truth for sync state.

All state transitions are coherent and atomic.

The UI never observes partial or impossible states.

The service remains encapsulated and testable.

Design Rules to Keep You Out of Trouble

Never mirror service state in multiple places.

Expose a single observable state or event stream from services.

Model state should always represent valid snapshots of the app.